

NDM: Event-Incidence Geometry as an External Memory for LLMs

Sep 27, 2026 · @Prabhat Saxena

Abstract

LLMs are capable reasoners, but they cannot hold everything in memory, and they cannot check what they say against what they were told. NDM (N-Dimensional Memory) is an external memory that supplies both. It records every event exactly, tracks how things change over time, and hands the LLM a structured picture of the past to reason over.

The model is geometric. Entities, dimensions and values are fixed pegs. Each event is a star of role strings pinned to its own tick on a timeline. Relationships are never stored: they come only from collisions, where events share a peg, and from trajectories, where an entity's events are ordered in time. The LLM supplies the interpretation; the engine preserves it without merging, guessing or overwriting.

Three epistemic rules keep memory true to its sources:

1. **Unresolved references become ambiguity buckets.** Each bucket holds its candidates and an open query, and is closed only by an explicit statement or the user.
2. **Beliefs are stored apart from facts, with an owner:** the user or the model. Beliefs never promote themselves.
3. **Dimensions drift, so the engine detects equivalence by footprint.** It collapses or relates dimensions only with confirmation.

The central hypothesis is about depiction. With the facts held constant, showing the LLM trajectories, collisions and open questions improves its understanding of change, causation and history, compared with retrieved chunks or flat fact lists.

Status: pre-research. A v0 reference ledger runs on PostgreSQL 16 and passes a 10-case adversarial suite. Every accuracy and performance claim is a hypothesis with a stated falsifier (Section 10).

1. Thesis: an external memory brain for the LLM

NDM does not try to make the LLM smarter. It supplies three things an LLM lacks: a memory that doesn't forget or blur, a sense of how things changed, and a way to check new statements against what it was told. The LLM already supplies intelligence.

What goes wrong without it:

- **Forgetting:** context windows end, and summaries drop detail.
- **Blurring:** vector retrieval returns similar text, not the right structure. "A acquired B" and "B acquired A" look alike.
- **Overwriting:** state memories replace old values, so "what did we believe before?" is lost.
- **Guessing:** "they approved it" becomes one specific person approving it.
- **Self-confirmation:** the model's own guess is repeated until it reads as fact.

What NDM covers:

Need	Mechanism
Exact recall	Every event stored as it was stated, on its own tick
Change over time	Trajectories on two clocks: when it happened, when it was learned
Relationships	Collisions on shared pegs, never pre-declared
Arbitration	Conflicts held open and flagged, attributed to their sources
Ambiguity vs belief	Buckets for the unresolved; owned beliefs for the inferred
Intent	Stated goals and plans as events; direction read from trajectories
Consistency checking	New statements checked against memory before they are trusted

Scope, stated once: NDM is exact relative to what the LLM extracted, not to the raw text, and it checks consistency with memory, not truth in the world. Extraction quality is therefore measured as a first-class variable.

2. Mental model: the switchboard

Picture a string-art board. Every entity, dimension and value is a fixed peg. Each event is a new tick on a timeline, with strings running from that tick to the pegs it touches. Memory is the board after thousands of events; understanding is reading the patterns the strings make.

- **Pegs never move and never merge by accident.** "PostgreSQL" and "MySQL" are different pegs. Identity comes from the peg, not from similar wording. The cost is honest: two pegs are not "near" each other unless evidence says so (Section 6).
- **An event is a star.** One tick at the centre, a few role strings out to pegs. Two events never share a tick, so their strings can never tangle. That rule is what keeps "AcmeCo acquired Globex" from blurring into "Initech acquired Hooli".
- **Collisions are pegs hit by more than one event.** A peg with strings from many ticks is a point where events touch.
- **Trajectories are one peg's strings read in time order.** Globex's trajectory is every event that touched Globex, oldest to newest. That sequence is its history, and its relationships.
- **Loose strings are ambiguity.** When a sentence says "they", the string ends in a bucket of candidate pegs, not on one peg.
- **Collapse and expansion are zoom.** From far away, each tick is a dot on the timeline. Zoom in and the full star reappears, because nothing was thrown away.

The board is the intuition. Section 4 gives its exact form, and Section 9 stores it as rows in a database, one row per string.

3. The event

An event is one happening, fully bound, pinned to its own tick, with nothing inferred inside it. Everything else in NDM is computed from events.

Part	Content	Rule
Tick	Unique, monotonic event number	One event per tick, never shared
Kind	acquisition, approval, statement...	From the catalog, like dimensions
Roles	dimension → entity peg or literal	1-6 roles, each atomic; direction lives in the role name

Part	Content	Rule
Occurred	A point, an interval, or vague ("last March")	World time, separate from the tick
Source	Speaker and provenance	Who said it, and where it came from
Status	Fact or belief (with owner), polarity, modality	One status per event
Open slots	An ambiguity bucket per unresolved role	Candidates plus a query; never a guess

Example: the user says on Friday, "They got acquired."

```
{
  "tick": 42,
  "kind": "acquisition",
  "roles": {
    "acquired": {"bucket": "b7", "query": "who was acquired?", "candidates":
["Globex", "Hooli"]},
    "acquirer": null
  },
  "occurred": "unspecified, before 2026-09-25",
  "source": {"speaker": "user", "provenance": "msg_881"},
  "status": {"type": "fact", "polarity": "pos", "modality": "certain"}
}
```

The event is a fact: an acquisition happened. Only the acquired role is unresolved. Ambiguity lives in a slot, not over the whole event.

Shape rules:

1. **One happening per event.** "AcmeCo bought Globex and fired the CEO" becomes two events on two ticks, related by their collision on Globex.
2. **Roles, not subject and object.** "Acquirer" and "acquired" survive passive voice, reordering and translation.
3. **No links to other events inside.** Relationships come only from collisions and trajectories.
4. **No inference inside a fact.** A conclusion is a separate belief event with an owner.
5. **Nesting only by reference.** "Alice said Bob approved it" is a statement event (speaker Alice) whose content role points to the approval event's peg.

Test of a well-shaped event: delete every other event, and this one still means exactly what the source said, no more and no less.

4. Formal model

The geometry is an incidence structure between pegs and events. Every operation below is exact, deterministic, and computed from stored events alone.

Pegs and events. Let E be the set of entity pegs, D the dimensions and V the literal values. Event k is a tuple of its tick, kind, role set, occurred time, source and status. Its role set pairs dimensions with entity pegs or values.

$$e_k = (k, \kappa_k, R_k, o_k, s_k, \sigma_k), \quad R_k \subseteq D \times (E \cup V)$$

Two clocks. The tick k orders events by when NDM learned them and gives each one a unique identity; this is what binds roles together. The occurred time o_k places the event in the world. Questions of the form "what did we know when" use k ; "what was true when" uses o_k . A fact learned today about last January gets a new tick and an old occurred time. No fractional ticks are needed.

Incidence matrices. Let B be the entity-by-event matrix with a 1 where entity i fills any role in event k . $B^{(d)}$ is the same matrix restricted to dimension d .

$$B_{ik} = \mathbb{1}[\exists d: (d, i) \in R_k], \quad B^{(d)}_{ik} = \mathbb{1}[(d, i) \in R_k]$$

Collisions, three levels.

Level	Definition	Meaning
Similarity	Events share a dimension or kind but no entity	Same kind of event; not related
Relationship	$(B^T B)_{kl} > 0$: events k and l share an entity	Both lie on that entity's trajectory
State interaction	Events share an entity in the same dimension, via $B^{(d)}$	Transition, repetition or conflict; triggers arbitration

Trajectories. An entity's trajectory is its row of B , read in order. Ordering by occurred time gives its history in the world; ordering by tick gives the order NDM learned it.

$$\tau(i) = \left(k : B_{ik} = 1 \right) \text{ ordered by } o_k \text{ (world) or } k \text{ (learning)}$$

Relationship strength. $(BB^T)_{ij}$ counts the events two entities share. Hub pegs such as the user touch almost everything, so shared events are down-weighted by how many entities each event touches, as in Adamic-Adar weighting. Proximity is then the weighted overlap of two trajectories, which is a similarity, not a metric; the paper claims nothing stronger.

$$w_{ij} = \sum_k \frac{B_{ik} B_{jk}}{\log(1 + \sum_m B_{mk})}$$

Paths. Indirect relationships chain collisions: entity to event to entity. Walks are typed (they follow only whitelisted dimensions) and bounded in depth. Their cost grows as branching raised to the depth, so the default depth is 2.

Collapse and expansion. Collapse maps a tick, or a run of ticks on a trajectory, to a summary line: kind, key pegs and flags. Expansion is a lookup of the stored events behind that line. Expansion is exact because nothing was discarded, not because the summary can be inverted.

5. Epistemics: facts, beliefs and ambiguity

NDM never lets a guess become a fact. Every event carries exactly one epistemic status, and each status has one owner and one way it can change.

Status	What it is	Owner	Enters "what happened"?
Fact	Explicitly stated, roles resolved	The speaker (user or source)	Yes
Ambiguity bucket	An unresolved role: candidates plus an open query	The engine	No, until resolved
User belief	"I think Rahul will block it"	The user, attributed	Shown as the user's belief

Status	What it is	Owner	Enters "what happened"?
Model belief	The model's view of something implicit	The model	Shown as the model's belief, never as fact

Ambiguity buckets. For "they got acquired", the engine opens a bucket with the query "who was acquired?". It fills the candidates by checking earlier events for collisions on the same dimension; here, entities recently in an acquired role. The model may attach a belief ("probably Globex, the most recent"), stored separately and linked to the bucket.

Resolution authority. A bucket closes only by an explicit statement or by the user. A strong implication ("Globex's new owners announced layoffs") strengthens a belief but cannot close the bucket. Unresolved buckets stay open indefinitely and are collapsed in depictions once they age.

No self-confirmation. The model restating its own belief is never evidence. A belief is promoted to fact only through the same resolution authority as a bucket. Without this rule, repetition turns guesses into facts: the exact corruption NDM exists to prevent.

State over time. The status fields keep distinct phenomena distinct:

Phenomenon	Representation	Effect
Absence	No event	Unknown, never false
Negation	polarity = neg	A claim that can conflict with a positive one
Retraction	A retract record targeting a specific event	Leaves state from its tick on; earlier replays still see it
Transition	A later occurred time on a single-valued dimension	Latest occurred time wins; history kept
Conflict	Different values tied at the latest occurred time	Both kept, flagged, attributed
Recurrence	The same fact on a new tick	More support and provenance; no count arithmetic

Dimensions are single-valued (status, owner) or multi-valued (approved_by, tags), and this cardinality decides transition versus conflict. The hard case: telling "it changed to rejected" from "it was rejected all along" is an extraction judgement, measured in Section 10 rather than assumed.

6. Dimension geometry

The LLM will not always use the same dimension for the same idea: acquired, was_bought, purchased_by. Collisions silently fail when pegs differ. NDM handles this in three layers.

- 1. Prevent it at extraction.** The dimension catalog goes into the extractor prompt: use an existing dimension if one fits; propose a new one only with a one-line definition.
- 2. Detect candidates by footprint.** A dimension's footprint is a vector describing what it connects: the entity types in its roles, its value distribution, the event kinds it appears in, and where it sits in trajectories. Unlike pegs, footprints are not one-hot, so similarity between them is meaningful. Two further signals help: dimensions that never co-occur in the same event are more likely to be the same thing, and name similarity from embeddings may contribute a hint, never a decision.

$$\text{close}(d_1, d_2) = \cos(\text{angle}(f(d_1), f(d_2))), \quad f(d) = \text{footprint vector of } d$$

3. Decide the relationship, not just merge or keep:

Relationship	Example	Effect
Equivalent	acquired = was_bought	Collapse into one canonical peg
Inverse	acquirer ↔ acquired_by	Linked, with direction preserved
Broader or narrower	acquired ⊂ ownership_change	Queries on the broad one include the narrow one
Related	acquired ~ merged_with	Shown together, never merged

Rules:

- Merges are a mapping layer, never a rewrite.** Events keep the dimension the LLM used. The alias is an append-only record, so a wrong merge can be undone without touching history.

- **Bias toward keeping dimensions separate.** A missed merge costs a missed collision. A false merge fuses two meanings and corrupts every trajectory that touches them.
- **Same authority rule as ambiguity.** The footprint proposes; an LLM judge or the user confirms. An unconfirmed near-match stays "related", not collapsed.

Measured by precision and recall of proposed relationships on a hand-labelled set of dimension pairs (Section 10).

7. Depiction: what the LLM sees

The product is the picture the LLM receives, not the store. The geometry decides which events to show and in what shape, so the model can reconstruct context instead of piecing together fragments.

Pipeline

1. **Seed:** link the question's mentions to pegs by exact alias, then lexical match, then embedding. This is the only fuzzy step, and its score is logged.
2. **Gather:** pull the seeds' trajectories, plus events one or two collisions away along whitelisted dimensions, with hub pegs down-weighted.
3. **Flag:** mark state interactions (transitions, conflicts), open buckets and beliefs touching the gathered events.
4. **Zoom:** keep recent and flagged events expanded; collapse older, quiet stretches of each trajectory into summary lines.
5. **Budget:** rank by collision weight, recency and flags, and trim to the token budget.
6. **Render:** one block per trajectory, with ticks and occurred times, so the model can cite and expand.

Example depiction

```
Globex trajectory (world time):
t1   [Mar] acquired by AcmeCo           (said by user)
t5   [Jun] CEO resigned                 (said by user)
t9   [Sep] acquired by Initech          CONFLICT with t1 on
"acquired"
t42  [before Sep 25] acquired – by ?     OPEN b7: who was acquired?
{Globex | Hooli}
      model belief: probably Globex (most recent)
      related via collisions: AcmeCo (t1), Initech (t9)
      collapsed: 14 earlier events, 2023–2025 [expand: t0–t14]
```

Agent contract. The system prompt tells the model to surface CONFLICT and OPEN items rather than silently pick, to say whose belief a belief is, and to cite ticks. It may ask to expand a collapsed range. Whether models honour this contract is itself tested (Section 10); if they ignore the flags, much of NDM's value disappears.

Consistency checking. Before an answer is sent, its factual claims can be extracted and checked against the same state. Claims that conflict with memory, or that assert the contents of an open bucket as fact, go back to the model with the conflicting ticks. This checks the answer against memory, not against the world.

8. Ingestion: the LLM extracts, the engine preserves

The LLM decides what a sentence means and packages it into events; the engine stores those events exactly and never merges what the LLM kept separate. Every fuzzy decision happens here, and each is recorded with provenance.

1. **Provenance:** hash the turn and record its source and the extractor version. A repeated (hash, extractor) pair does nothing, so re-ingestion is idempotent.
2. **Extraction:** one schema-constrained LLM call per turn returns events in the Section 3 shape. The dimension catalog and recent pegs are in the prompt. The model must split compound sentences into separate events, give speaker, polarity and modality, and return candidates plus a query for any unresolved role. The call is model-agnostic; a local model on SGLang or vLLM is the default.
3. **Coreference:** map mentions to pegs by alias table, then candidate retrieval, then an LLM decision. A tie becomes a bucket, never a guess. New aliases are recorded with provenance.
4. **Dimension check:** map raw names through confirmed aliases. Unknown dimensions go to quarantine, and their footprints feed Section 6.
5. **Classification:** label each event as new, recurrence, transition, correction, or resolution of an open bucket (explicit statement or user only).
6. **Commit:** assign the next tick to each event and write it in one transaction. Database constraints enforce shape, value types and append-only writes.

Cost budget (target, unmeasured): one extractor call per turn, plus at most one coreference call on ties. Writes run asynchronously on a queue, so replies never wait on memory. Reads are synchronous, with a target under 100 ms p95 at 10^5 events per user.

9. Reference implementation

The board is stored as rows: one row per string. v0 below is tested on PostgreSQL 16 against 10 adversarial cases. v1 adds the parts of the model defined after v0 was written.

How v0 maps to the model: an event peg is a symbol of kind "event". Each role string is a statement row whose subject is the event peg and whose attribute is the role. valid_from and valid_to hold occurred time; tx_time is the learning clock. Ambiguity groups and resolutions implement buckets.

Test	Checks	Result
T1	Role inversion via separate event pegs stays distinct	Pass
T2	pending → approved is a transition, not a conflict	Pass
T3	Two speakers, same occurred time, different values → both flagged, attributed	Pass
T4	Open bucket excluded from state; resolution visible only after it is recorded; non-candidate resolution rejected	Pass
T5	Retracted fact gone now, visible when replaying earlier	Pass
T6	Negation surfaces as a conflict, not a deletion	Pass
T7	Late-learned fact about the past appears only after it was learned	Pass
T8	Quarantined (hallucinated) dimension rejected at write	Pass
T9	UPDATE on the ledger rejected	Pass
T10	Typed 2-hop walk finds Globex; the wrong dimension set does not	Pass

v1 changes (week 1 of the plan):

- An event table with a unique monotonic tick (bigserial), kind, occurred interval plus a free-text form for vague times, source, and status. Role statements reference it.

- An owner field: fact, user belief, or model belief. A constraint stops model beliefs from counting as evidence toward resolution.
- A query text on each bucket, and a resolution-authority field (explicit statement or user) with a check constraint.
- Dimension relationships (equivalent, inverse, broader, related), with confirmation records.
- Incidence views (B and the per-dimension $B^{(d)}$) and a trajectory function ordered by either clock.
- Tests T11–T15: attribution isolation, modality exclusion, re-ingestion idempotency, no self-confirmation, and the resolution-authority check.

v0 schema (tested):

```
-- NDM reference schema (PostgreSQL 16). Append-only, bitemporal statement
ledger.
CREATE TYPE symbol_kind AS ENUM ('entity', 'event', 'attribute');
CREATE TYPE value_type AS ENUM ('ref', 'literal');
CREATE TYPE cardinality AS ENUM ('one', 'many');
CREATE TYPE stmt_op AS ENUM ('assert', 'retract');
CREATE TYPE polarity AS ENUM ('pos', 'neg');
CREATE TYPE modality AS ENUM ('certain', 'probable', 'possible',
'hypothetical');
CREATE TYPE amb_axis AS ENUM ('subject', 'value', 'attribute',
'valid_time');

CREATE TABLE provenance (
    provenance_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    source_ref text NOT NULL, -- conversation / message id
    source_hash text NOT NULL, -- hash of the source chunk
    source_text text,
    extractor text NOT NULL, -- model + prompt version
    extracted_at timestamptz NOT NULL DEFAULT clock_timestamp(),
    UNIQUE (source_hash, extractor) -- idempotent re-ingestion
);

CREATE TABLE symbol ( -- entities, reified events,
meta-attributes
    symbol_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    kind symbol_kind NOT NULL,
    label text NOT NULL, -- NOT unique: two different
"Alice"s are two symbols
```

```

    created_tx timestampz NOT NULL DEFAULT clock_timestamp()
);

CREATE TABLE symbol_alias (                                -- coreference decisions, with
lineage
    alias          text NOT NULL,
    symbol_id      uuid NOT NULL REFERENCES symbol,
    provenance_id  uuid NOT NULL REFERENCES provenance,
    tx_time        timestampz NOT NULL DEFAULT clock_timestamp(),
    PRIMARY KEY (alias, symbol_id)
);

CREATE TABLE attribute (                                    -- the schema catalog
    name          text PRIMARY KEY,
    value_type    value_type NOT NULL,
    cardinality    cardinality NOT NULL,
    status        text NOT NULL DEFAULT 'canonical' CHECK (status IN
('canonical', 'quarantined')),
    description    text
);

CREATE TABLE attribute_alias (                             -- synonym map: a lookup table,
not a matrix
    raw          text PRIMARY KEY,
    canonical    text NOT NULL REFERENCES attribute(name)
);

CREATE TABLE ambiguity_group (
    group_id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    axis          amb_axis NOT NULL,
    provenance_id uuid NOT NULL REFERENCES provenance,
    tx_time       timestampz NOT NULL DEFAULT clock_timestamp()
);

CREATE TABLE statement (
    statement_id  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    op            stmt_op NOT NULL DEFAULT 'assert',
    retracts      uuid REFERENCES statement,          -- set iff op =
'retract'
    subject_id    uuid REFERENCES symbol,
    attribute     text REFERENCES attribute(name),
    value_ref     uuid REFERENCES symbol,
    value_lit     text,

```

```

    polarity          polarity NOT NULL DEFAULT 'pos',      -- negation: "X is
false"
    modality          modality NOT NULL DEFAULT 'certain',
    speaker_id        uuid REFERENCES symbol,              -- attribution: who
asserted it
    event_id          uuid REFERENCES symbol,              -- context event
(optional)
    ambiguity_group_id uuid REFERENCES ambiguity_group,
    confidence         real NOT NULL DEFAULT 1.0 CHECK (confidence BETWEEN 0
AND 1),
    valid_from        timestampz,
    valid_to          timestampz,
    tx_time           timestampz NOT NULL DEFAULT clock_timestamp(),
    provenance_id      uuid NOT NULL REFERENCES provenance,
    CHECK (
        (op = 'retract' AND retracts IS NOT NULL AND subject_id IS NULL AND
attribute IS NULL
            AND value_ref IS NULL AND value_lit IS NULL AND ambiguity_group_id IS
NULL)
        OR
        (op = 'assert' AND retracts IS NULL AND subject_id IS NOT NULL AND
attribute IS NOT NULL
            AND valid_from IS NOT NULL AND ((value_ref IS NULL) <> (value_lit IS
NULL))))
    ),
    CHECK (valid_to IS NULL OR valid_to > valid_from)
);

CREATE TABLE ambiguity_resolution (      -- latest resolution per group
wins; history kept
    resolution_id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    group_id           uuid NOT NULL REFERENCES ambiguity_group,
    chosen_statement_id uuid REFERENCES statement,          -- NULL = none of the
candidates
    provenance_id      uuid NOT NULL REFERENCES provenance,
    tx_time            timestampz NOT NULL DEFAULT clock_timestamp()
);

CREATE INDEX stmt_state_idx ON statement (subject_id, attribute, valid_from
DESC) WHERE op = 'assert';
CREATE INDEX stmt_ref_idx  ON statement (value_ref) WHERE value_ref IS NOT
NULL;
CREATE INDEX stmt_retr_idx ON statement (retracts) WHERE op = 'retract';

```

```

CREATE INDEX stmt_amb_idx    ON statement (ambiguity_group_id) WHERE
ambiguity_group_id IS NOT NULL;
CREATE INDEX stmt_tx_idx     ON statement (tx_time);
CREATE INDEX alias_idx       ON symbol_alias (lower(alias));

-- Append-only guard. Erasure (privacy) is an explicit, audited path, not an
UPDATE.
CREATE FUNCTION forbid_mutation() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF current_setting('ndm.allow_erasure', true) = 'on' THEN
        IF TG_OP = 'DELETE' THEN RETURN OLD; END IF;
        RETURN NEW;
    END IF;
    RAISE EXCEPTION 'NDM ledger is append-only (% on %)', TG_OP, TG_TABLE_NAME;
END $$;
CREATE TRIGGER statement_append_only BEFORE UPDATE OR DELETE ON statement
    FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
CREATE TRIGGER resolution_append_only BEFORE UPDATE OR DELETE ON
ambiguity_resolution
    FOR EACH ROW EXECUTE FUNCTION forbid_mutation();

-- Write-time validation: catalog status, value type, retraction target.
CREATE FUNCTION validate_statement() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE a attribute%ROWTYPE;
BEGIN
    IF NEW.op = 'retract' THEN
        IF NOT EXISTS (SELECT 1 FROM statement WHERE statement_id = NEW.retracts
AND op = 'assert') THEN
            RAISE EXCEPTION 'retract must target an existing assertion';
        END IF;
        RETURN NEW;
    END IF;
    SELECT * INTO a FROM attribute WHERE name = NEW.attribute;
    IF a.status <> 'canonical' THEN
        RAISE EXCEPTION 'attribute % is quarantined', NEW.attribute;
    END IF;
    IF (a.value_type = 'ref') <> (NEW.value_ref IS NOT NULL) THEN
        RAISE EXCEPTION 'attribute % expects a % value', NEW.attribute,
a.value_type;
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER statement_validate BEFORE INSERT ON statement

```

```

FOR EACH ROW EXECUTE FUNCTION validate_statement();

CREATE FUNCTION validate_resolution() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.chosen_statement_id IS NOT NULL AND NOT EXISTS (
        SELECT 1 FROM statement WHERE statement_id = NEW.chosen_statement_id
            AND ambiguity_group_id = NEW.group_id) THEN
        RAISE EXCEPTION 'chosen statement is not a candidate of group %',
NEW.group_id;
    END IF;
    RETURN NEW;
END $$;

CREATE TRIGGER resolution_validate BEFORE INSERT ON ambiguity_resolution
FOR EACH ROW EXECUTE FUNCTION validate_resolution();

-- Q1: what the system held as asserted, as known at transaction time
known_at.
CREATE FUNCTION live_statements(known_at timestamptz DEFAULT 'infinity')
RETURNS TABLE (statement_id uuid, subject_id uuid, attribute text, value_ref
uuid, value_lit text,
                polarity polarity, modality modality, speaker_id uuid,
event_id uuid,
                ambiguity_group_id uuid, confidence real, valid_from
timestamptz,
                valid_to timestamptz, tx_time timestamptz, provenance_id uuid,
amb_status text)
LANGUAGE sql STABLE AS $$
WITH res AS (
    SELECT DISTINCT ON (r.group_id) r.group_id, r.chosen_statement_id
    FROM ambiguity_resolution r
    WHERE r.tx_time <= known_at
    ORDER BY r.group_id, r.tx_time DESC
)
SELECT s.statement_id, s.subject_id, s.attribute, s.value_ref, s.value_lit,
s.polarity,
    s.modality, s.speaker_id, s.event_id, s.ambiguity_group_id,
s.confidence,
    s.valid_from, s.valid_to, s.tx_time, s.provenance_id,
CASE WHEN s.ambiguity_group_id IS NULL THEN 'none'
    WHEN res.group_id IS NULL THEN 'open'
    WHEN res.chosen_statement_id = s.statement_id THEN 'chosen'
    ELSE 'rejected' END
FROM statement s

```

```

LEFT JOIN res ON res.group_id = s.ambiguity_group_id
WHERE s.op = 'assert' AND s.tx_time <= known_at
      AND NOT EXISTS (SELECT 1 FROM statement x
                      WHERE x.op = 'retract' AND x.retracts = s.statement_id
                      AND x.tx_time <= known_at)
$$;

-- Q2: believed state at valid time valid_at, as known at known_at.
-- Cardinality 'one': latest valid_from per (subject, attribute) wins
(supersession).
-- Cardinality 'many': latest valid_from per (subject, attribute, value).
-- Distinct (value, polarity) pairs tied at the winning valid_from =
contradiction.
-- Open or rejected ambiguity candidates and weak modalities are excluded
(surfaced elsewhere).
CREATE FUNCTION state_at(valid_at timestampz DEFAULT now(), known_at
timestampz DEFAULT 'infinity')
RETURNS TABLE (subject_id uuid, attribute text, value_ref uuid, value_lit
text, polarity polarity,
                statement_id uuid, speaker_id uuid, valid_from timestampz,
in_conflict boolean)
LANGUAGE sql STABLE AS $$
WITH c AS (
    SELECT l.*,
           CASE WHEN a.cardinality = 'one' THEN ''
                ELSE COALESCE(l.value_ref::text, 'lit:' || l.value_lit) END
AS vkey
FROM live_statements(known_at) l
JOIN attribute a ON a.name = l.attribute
WHERE l.amb_status IN ('none', 'chosen')
      AND l.modality IN ('certain', 'probable')
      AND l.valid_from <= valid_at
      AND (l.valid_to IS NULL OR l.valid_to > valid_at)
), ranked AS (
    SELECT c.*, rank() OVER (PARTITION BY c.subject_id, c.attribute, c.vkey
                            ORDER BY c.valid_from DESC) AS rk
FROM c
), top AS (
    SELECT * FROM ranked WHERE rk = 1
), conflicts AS (
    SELECT t.subject_id, t.attribute, t.vkey,
           count(DISTINCT COALESCE(t.value_ref::text, 'lit:' || t.value_lit)
|| '|' || t.polarity::text) > 1 AS in_conflict

```

```

    FROM top t GROUP BY t.subject_id, t.attribute, t.vkey
  )
  SELECT t.subject_id, t.attribute, t.value_ref, t.value_lit, t.polarity,
         t.statement_id, t.speaker_id, t.valid_from, k.in_conflict
  FROM top t
  JOIN conflicts k USING (subject_id, attribute, vkey)
  $$;

```

```

-- Q3: unresolved ambiguity, as known at known_at.
CREATE FUNCTION open_ambiguities(known_at timestampz DEFAULT 'infinity')
RETURNS TABLE (group_id uuid, axis amb_axis, candidates uuid[])
LANGUAGE sql STABLE AS $$
  SELECT g.group_id, g.axis, array_agg(l.statement_id ORDER BY
l.statement_id)
  FROM live_statements(known_at) l
  JOIN ambiguity_group g ON g.group_id = l.ambiguity_group_id
  WHERE l.amb_status = 'open'
  GROUP BY g.group_id, g.axis
  $$;

```

```

-- Q4: typed traversal over reference-valued statements, cycle-safe, depth-
bounded.
-- Cost is O(branching^depth) in the worst case: keep depth small and
attributes typed.
CREATE FUNCTION traverse(seed uuid, allowed_attrs text[] DEFAULT NULL,
max_depth int DEFAULT 2,
                        known_at timestampz DEFAULT 'infinity')
RETURNS TABLE (symbol_id uuid, depth int, path uuid[], via_statements uuid[])
LANGUAGE sql STABLE AS $$
  WITH RECURSIVE e AS (
    SELECT l.subject_id AS a, l.value_ref AS b, l.statement_id AS sid
    FROM live_statements(known_at) l
    WHERE l.value_ref IS NOT NULL AND l.polarity = 'pos'
      AND l.amb_status IN ('none', 'chosen')
      AND (allowed_attrs IS NULL OR l.attribute = ANY (allowed_attrs))
  ), u AS (
    SELECT a, b, sid FROM e UNION ALL SELECT b, a, sid FROM e
  ), walk AS (
    SELECT seed AS node, 0 AS d, ARRAY[seed] AS p, ARRAY[]::uuid[] AS v
    UNION ALL
    SELECT u.b, w.d + 1, w.p || u.b, w.v || u.sid
    FROM walk w JOIN u ON u.a = w.node
  )

```

```
WHERE w.d < max_depth AND NOT (u.b = ANY (w.p))
)
SELECT DISTINCT ON (node) node, d, p, v FROM walk ORDER BY node, d
$$;
```

Known performance limit: live_statements scans the ledger on every call. Beyond roughly 10⁶ statements (an estimate, to be measured), add a materialised current-state table maintained on insert, and keep these functions for as-of queries.

10. Evaluation

The headline experiment isolates the geometry. Hold the extracted events fixed, change only how they are depicted, and measure what the LLM understands.

Experiment 1: depiction, facts held constant. Give the same answering model the same token budget in three conditions:

Condition	What the model receives
A. Retrieved chunks	Raw source passages ranked by embedding similarity (standard RAG)
B. Flat facts	The same extracted events as an unordered list (a typical memory store)
C. NDM depiction	Trajectories, collision flags, open buckets, owned beliefs, zoom (Section 7)

Extraction is identical across B and C, so any gain from C over B comes from the geometry and depiction alone. Questions cover five types: change ("what changed about X?"), causation and sequence ("what happened after X?"), history ("what did we believe in June?"), conflict ("who says what about X?"), and ambiguity ("who was acquired?", where the correct answer is "unresolved").

Hypotheses:

ID	Hypothesis	Metric	Falsified if
H1	Depiction C beats B and A on change, sequence and history questions	Accuracy, 3 seeds, paired bootstrap	C is not better than B beyond noise

ID	Hypothesis	Metric	Falsified if
H2	C reduces silent wrong picks on conflict and ambiguity questions	Silent wrong-pick rate; correct "unresolved" rate	Not lower than B
H3	Degradation is graceful under extraction errors	Accuracy under injected errors: dropped events, swapped roles, spurious dimensions (10/20/30%)	C's curve falls as fast as B's or faster
H4	Transition vs correction is classifiable at ingestion	F1 on 300 hand-labelled cases	F1 below 0.8
H5	Dimension relationships are detectable by footprint	Precision and recall on labelled dimension pairs	Precision below 0.9 at useful recall
H6	No self-confirmation holds in long runs	Model-belief $\rightarrow$ fact promotions without an explicit statement or user	Any promotion

Datasets: a purpose-built corpus from long multi-party conversations with conflicts, pronoun ambiguity and late-learned facts, with gold events and answers. Then LongMemEval and LoCoMo for external comparability. Their category names should be verified against the current releases before citing.

Baselines and ablations: Mem0 and Graphiti/Zep as full systems, and full context where it fits. NDM ablations: no buckets (forced guess), no zoom (everything expanded), no flags, no agent contract, tick-only (no occurred time).

What does not count as evidence: role-inversion tests (they test the extractor) and demonstrations that the engine traced a path (they pass by construction).

11. Related work

The individual pieces are not new. Immutable fact logs, bitemporality and entity-linked events all exist. NDM's claim is the combination, and its evaluation. The table is written from memory and must be checked against current papers and documentation before submission.

System	What it already does	What NDM adds or tests
Datomic	Immutable datoms (e, a, v, tx, op); per-attribute cardinality; retraction as a datom	Valid time; ambiguity groups; attribution and modality; LLM context assembly
XTDB	Bitemporal (valid + transaction time) document store	Statement-level epistemics; agent-memory evaluation
Graphiti / Zep	Temporal knowledge graph for agents; edges carry validity intervals; newer facts invalidate conflicting edges	Keep conflicts and ambiguity open instead of auto-invalidating; explicit resolution records
Mem0	LLM decides add / update / delete over extracted memories	Nothing is deleted; updates are supersession with history
Letta (MemGPT)	Tiered memory the agent edits itself	Deterministic ledger under a fuzzy agent; the agent can cite statement IDs

Event-centric and temporal knowledge graphs already link events through shared entities, so collisions alone are not the contribution. **What NDM claims as its own:**

- relationships derived only from collisions and trajectories, at three explicit levels
- per-event ticks as the binding mechanism, with two clocks
- ambiguity buckets with a strict resolution authority, and owned beliefs with no self-confirmation
- footprint-based dimension geometry with confirmed collapse
- a depiction layer designed for an LLM's context window, tested with facts held constant

The risk is that this proves different but not better. Section 10 is built to find out.

12. Limitations and open questions

The two largest risks are extraction quality and whether models act on the structure they are shown.

- **Extraction ceiling:** every guarantee is relative to extracted events. A wrong event is preserved perfectly.

- **Models may ignore flags:** if the LLM resolves conflicts or buckets on its own despite the contract, the depiction's value shrinks. Ablations measure this.
- **Vague occurred times:** "last March" and "recently" need interval semantics. Conflict detection across overlapping vague intervals is open.
- **Speaker trust:** a later claim from a low-trust speaker currently counts like any other. Arbitration needs a trust policy. Open: is it configured or learned?
- **Hub pegs:** down-weighting by event size is a heuristic. Open: is it enough for users who dominate their own memory?
- **Stale gaps in trajectories:** if a late-learned fact covers March–April and nothing covers May, the model must not assume the March value still holds. Open: depict the gap explicitly?
- **Erasure versus immutability:** append-only memory conflicts with the right to erasure. Crypto-shredding of source text plus audited deletion must be designed before storing real user data.
- **Scale:** v0 recomputes state from the full ledger. Materialised trajectories and partitioning by user are needed before load tests.

13. Research plan

The plan runs ten weeks to a go/no-go on H1 and H2. That is realistic for one lead alongside a day job; eight would be optimistic. Experiment 1 comes before the public benchmarks, because it isolates the geometry.

Weeks	Deliverable	Gate
1	v1 schema (event table, ticks, owners, bucket queries, dimension relationships) in CI with T1–T15	All tests green
2	Extractor v1: event-shaped output, prompt, attribute catalog for one domain (engineering team chat)	Extraction P/R measured on 50 hand-labelled turns
3	Coreference and classification steps; labelled set of 300 transition/correction cases	H4 F1 reported

Weeks	Deliverable	Gate
4	Depiction layer: trajectories, flags, zoom, budget; FastAPI plugin API (ingest, depict, replay)	End-to-end demo on one long conversation
5	Purpose-built corpus: gold events and answers across the 5 question types	Corpus frozen; second-annotator agreement checked
6	Experiment 1: conditions A, B, C on the corpus	H1 and H2 results, 3 seeds
7–8	Ablations; error sweep (H3); dimension pairs (H5); LongMemEval, LoCoMo, Mem0 and Graphiti	Results table with confidence intervals
9–10	Write-up: results, negative results, decision	Go if H1 or H2 holds; otherwise narrow scope or stop

Build the corpus in week 5 even if weeks 2–4 slip. Without Experiment 1, the rest is design.

Appendix A. The original NDM specification: keep, fix, remove

The original tensor specification had the right intuitions and the wrong mathematics for several of them. Section numbers below refer to that document.

Original §	Idea	Decision	Where it lives now
2	Entity $\times$ Dimension $\times$ Value $\times$ Time	Keep; T becomes a unique event tick plus occurred time	Sections 3, 4
3–6	Atomic values; values and dimensions can be entities; shared registry	Keep	Sections 3, 6
1, 25, 26	Projection, point P(E,t), trajectory	Keep	Section 4
16–19	Recurrence on new ticks; contradictions held open; additive memory	Keep	Section 5

Original §	Idea	Decision	Where it lives now
27–29, 37–38	Collisions create relationships; no stored edges	Keep, with three collision levels	Section 4
30–36	Collapse and expansion	Keep as zoom; drop "exact inverse"	Sections 4, 7
10–11, 60–62	Coreference before storage; provenance; exact relative to extraction	Keep; ties go to buckets	Sections 5, 8
41, 58	Typed paths; time windows	Keep	Sections 4, 7
12–15	One coefficient for presence, count, uncertainty, negation	Fix: separate status fields	Section 5
20–23	Shared and fractional ticks	Fix: one tick per event, two clocks	Section 4
39, 42, 59	Adjacency powers	Fix: incidence matrices B , BB^T , B^TB	Section 4
43–45	"Manhattan" $1/\text{count}$ as a metric	Fix: weighted trajectory overlap, called a similarity	Section 4
48–53	0.5/0.5 fractional superposition	Fix: buckets with candidates, query, resolution authority	Section 5
7–8	Surjective alias matrix	Fix: catalog, footprints, confirmation	Section 6
Decay processor	$\exp(-\lambda\Delta t)$ fading	Fix: ranking in depiction only; storage never fades	Section 7
14	Multiplicity within a tick	Remove: impossible with one event per tick	—
46–47	Spherical normalisation	Remove: no effect on one-hot pegs	—
57	Variance scan for ambiguity	Remove: buckets are queried directly	—

Original §	Idea	Decision	Where it lives now
63	Hallucinated dimensions "safe at 90°"	Remove: quarantine instead	Section 8

Appendix B. Corrections to the NDL draft

Each row maps a claim in the NDL draft to its fix in this paper. The draft's other Q&A answers are either absorbed into Sections 3–5 or were restatements.

Draft item	Problem	Fix here
Title and name "Non-Dimensional Ledger"	Meaningless name; implies a geometric model	NDM, a bitemporal epistemic ledger
Q7 "Surjective Dictionary Transformation Mapping Matrix"; Q20 "Bitemporal Matrix Coordinate Index"; Q51 "Disambiguation Assertion Matrix Row"; Q96 "Normalized Subtraction Join"	Leftover tensor vocabulary for plain lookups and rows	Plain terms: alias table, valid/tx time, resolution row
Q15, Q89: negation = retract	Conflates "X is false" with "withdraw the claim X"	polarity = neg vs a retract row targeting a statement ID (T5, T6)
Q18 vs Q19; view_active_state	Every state change flagged as a contradiction	Attribute cardinality plus supersession by valid_from (T2, T3)
Ambiguity rows stored as plain asserts; no resolution link	"They approved" becomes both approved; no link from resolution to group	Open candidates excluded; ambiguity_resolution table (T4)
tx_time_end on an "immutable" ledger	Requires UPDATE	Pure append with tx_time only; triggers block UPDATE and DELETE (T9)

Draft item	Problem	Fix here
Single valid_time point; Q58 filter	Half-bitemporal; no as-of-knowledge query	valid_from/valid_to plus state_at(valid_at, known_at) (T7)
Q29, Q92, Q94: event reification	No event linkage in the schema	Event symbols with role attributes; event_id; speaker_id (T1)
Q90–93: modality and belief namespaces	Columns absent; the namespace approach doubles attributes	modality enum plus speaker_id; attributes stay single
No value integrity	value_entity_id and value_literal could both be set or both null	CHECK exactly one, plus a value-type trigger
Q63 vs Q5	Strict validation contradicts dynamic attributes	Quarantine status; review before promotion (T8)
Q64 confidence multiplication	Contradicts the boolean domain (Q12)	Confidence used for ranking only
Q84 complexity	O(1) B-tree lookup and linear traversal are false	O(log n) lookups; O(branching^depth) traversal with cycle guard
Q62 "(0.0)"; Q83 "tensor row"	Tensor residue	Removed
Q69, Q100 falsifiers	Test the extractor or pass by construction	H1–H6 with baselines and ablations (Section 10)
Q72–73 novelty	Datomic, XTDB and Graphiti already do retraction and bitemporality	Novelty narrowed to epistemic layer plus evaluation (Section 11)
Schema SQL	Comments fused into statements; would not run	Tested on PostgreSQL 16 (Section 9)
Missing product layer	No retrieval, serialisation or token budget	Section 7